

이 력 서

전 소 현

프론트엔드 개발자



이메일 tamizy@naver.com GitHub @laven-ia

96%

Painting Time 단축
4,300ms → 170ms

86%

Main Thread Long Task 감소
51회 → 7회

500줄

API 관련 코드 감축
TanStack Query 도입

01 소개

React와 TypeScript를 기반으로 실시간 관제 시스템과 디지털 트윈 기반 데이터 시각화 서비스를 개발해 온 **2년 차 프론트엔드 개발자**입니다. UI/UX 설계부터 실시간 데이터 시각화, 영상 스트리밍, Electron 통합 배포와 일부 백엔드 API 유지보수까지 제품 전반의 개발을 담당해 왔습니다.

24시간 운영 환경의 메모리 증가 문제를 분석해 **화이트 스크린 문제를 해결**하고, TanStack Query 기반으로 **API 구조를 재설계해 관련 코드를 약 500줄 줄였습니다**. 고해상도 지도에서는 렌더링 구조를 최적화해 **Painting Time을 4,300ms에서 170ms로 96% 단축**하고, **Main Thread Long Task를 51회에서 7회로 86% 감소**시켰습니다.

사용자 경험뿐 아니라 실제 운영 안정성과 유지보수성을 함께 고려하며, 문제를 구조적으로 분석하고 개선하는 개발을 지향합니다.

02 기술 스택

FRONTEND (CORE)

React TypeScript JavaScript
Vite

FRONTEND (EXPERIENCE)

Next.js SvelteKit PyQt5
NiceGUI

STATE & DATA FETCHING

TanStack Query Redux Toolkit
Context API

NETWORK & REAL-TIME

REST API WebSocket

VISUALIZATION

Canvas API ECharts Three.js
Unity WebGL

MEDIA STREAMING

RTSP HLS MediaMTX
FFmpeg

DESKTOP & DEPLOYMENT

Electron PyInstaller

BACKEND

FastAPI Python

ADDITIONAL

Unity QGIS glTF Git Jira Figma

03 주요 프로젝트

전기차 화재 감지 시스템

2025.05 - 현재

Frontend Developer

담당 UI/UX 설계 및 프론트엔드 개발, 영상 스트리밍·통합 배포 구조 설계, 백엔드 API 유지보수

팀 구성 Frontend 1명, Backend 3명

프로젝트 개요

- 가시광·열화상 카메라와 이기종 센서를 통합한 실시간 화재 감지·관제 시스템
- 영상·온도 데이터를 기반으로 ROI 관리, 이상 온도 알람 및 화재·연기·사람 탐지 결과 시각화

담당 역할 및 주요 성과

- 24시간 상시 운영 환경에서 발생한 백화 현상을 JavaScript Heap-Electron renderer 메모리 및 렌더링 경로까지 확장해 분석하고, 24시간 미만 주기로 발생하던 화이트 스크린 현상과 메모리 이상향 문제 해소
- JavaScript → TypeScript 전환과 컴포넌트 책임 분리를 진행하고 WebSocket Context를 역할별로 분리하여, 타입 관련 런타임 오류 4건 → 0건으로 개선 및 불필요한 리렌더링 최소화
- 메인 페이지에 집중된 9개 도메인 API 호출을 types → services → hooks/queries 구조로 분리하고 TanStack Query를 도입해 API 관련 코드 약 500줄 감축, 중복 요청·직접 상태 동기화 로직 제거
- 바닥과 천장, 두 개의 관제 플랫폼을 단일 관제 플랫폼으로 통합하고, 카메라·오프가스·불꽃 센서 데이터를 WebSocket으로 수신해 실시간 시각화
- 복수 센서 데이터를 종합 위험도 지표로 구성해 단일 센서 오작동에 따른 오인 알람 가능성 완화
- Canvas 기반 폴리곤 ROI 생성·편집 기능과 탐지 객체 BBox 실시간 렌더링 구현
- MediaMTX-FFmpeg 기반 RTSP → HLS 영상 스트리밍 파이프라인 설계
- 영상 장비가 없는 특수 현장 관제를 위해 공공 데이터 기반 지형·건물 데이터를 QGIS에서 좌표 정합·후처리한 뒤, glTF로 변환해 Three.js 기반 3D 관제 화면 구현

TECH STACK

React

TypeScript

JavaScript

TanStack Query

Canvas API

ECharts

WebSocket

REST API

Electron

MediaMTX

FFmpeg

교통 신호 운영 분석 소프트웨어

2025년 07월 - 2026년 01월

Frontend Developer

담당 UI/UX 설계, 프론트엔드·Unity 개발, FCD 데이터 파이프라인 구현

팀 구성 Frontend 1명, Backend 2명

프로젝트 개요

- SUMO·Unity·React를 연동한 디지털 트윈 기반 실시간 교통 시뮬레이션 시스템
- 교차로 신호 최적화 전·후의 교통 흐름을 3D 시뮬레이션하고 성능 비교·통계 시각화 제공

담당 역할 및 주요 성과

- 고해상도 지도 렌더링에 ImageBitmap 캐싱·Multi-Canvas·오프스크린 밍매핑을 적용해 지도 로딩 시간 약 10초 → 3초, Painting Time 약 4,300ms → 170ms(96% 감소)로 개선
- 지도 드래그·줌 과정의 렌더링 병목을 최적화해 메인 스레드 Long Task 51회 → 7회(86% 감소)로 축소
- 수 GB 규모 FCD 데이터를 Python 기반 NDJSON 온디맨드 스트리밍 구조로 전환하고 시간·바이트 인덱싱을 적용해 처리 시간을 400초 → 85초(78.7% 단축)로 개선, Java 백엔드에서 활용 가능한 구조로 인계

- SUMO 차량 위경도 데이터를 Unity 좌표계로 변환하는 호모그래피 기반 좌표 변환 로직을 구현하여 실시간 3D 차량 시각화 정확성 확보
- 교차로 선택 → 모델 생성 → 최적화 요청 → 이력 조회 → 전후 성능 비교로 이어지는 분석 대시보드 구현
- Unity(C#)에서 싱글톤·옵저버 패턴 기반 WebSocket 구조를 설계해 실시간 FCD 데이터를 각 시각화 모듈로 전달
- Unity WebGL 빌드를 React 애플리케이션에 임베드해 신호 최적화 전후의 3D 교통 시뮬레이션 제공
- 공공 지형·건물 데이터를 QGIS에서 가공하고 glTF로 변환해 Unity 시뮬레이션 환경에 적용

TECH STACK

React

TypeScript

Redux Toolkit

Canvas API

Recharts

Unity WebGL

C#

WebSocket

Python

NDJSON

QGIS

Electron

04 경력 및 학력

경력사항(개발)

2025년 05월 - 현재

시티아이랩(주)
프론트엔드 개발자

경력사항(기타)

2024년 11월 - 2025년 05월

(주)누리라인
전장·배선 설계 엔지니어

2020년 09월 - 2021년 11월

(주)누리라인
전장·배선 설계 엔지니어

학력사항

2016년 03월 - 2021년 02월

한국기술교육대학교
전자공학과 학사

2013년 03월 - 2016년 02월

청주대성고등학교

교육 및 활동

2023년 07월 - 2024년 06월

삼성 청년 SW 아카데미 10기 (SSAFY)

05 수상 · 자격증 · 어학

수상

2023년 11월

SSAFY 1학기 프로젝트 최우수상

자격증

2009년 03월 20일

DIAT-프리젠테이션
고급(194점)

2009년 08월 14일

DIAT-워드프로세서
고급(198점)

어학

-

일본어

간단한 회화 가능

위에 기재한 사항은 사실과 틀림이 없습니다.

2026년 08월 13일

성명 : 전 소 현 (인) 전소현